

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Лабораторная работа №6.

Тема: «АДТ. Контейнеры»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 13

Выполнила: студентка группы РИС-25-26

Лаптева Елена Владимировна

Принял: доц. Полякова О.А.

Пермь, 2026

Постановка задачи:

1. Определить класс-контейнер (список с целочисленными ключами).
2. Реализовать:
 - a. Конструкторы (по умолчанию, с параметрами, копирования)
 - b. Деструктор
 - c. Операции ввода-вывода (`operator<<`, `operator>>`)
 - d. Операцию присваивания (`operator=`)
3. Перегрузить операции, указанные в варианте (см. ниже).
4. Реализовать класс-итератор для последовательного доступа к элементам списка:
 - a. Операции инкремента/декремента (`++`, `--`)
 - b. Разыменования (`*`)
 - c. Сравнения (`==`, `!=`)
5. Написать тестирующую программу, демонстрирующую:
 - a. Создание объектов
 - b. Работу всех перегруженных операций
 - c. Работу с итератором

Класс-контейнер: СПИСОК (LIST)

Ключевые значения типа `int`.

Реализовать операции:

1. `operator[]` — доступ по индексу (возвращает элемент по индексу `i`)
2. `operator+` (вектор) — сложение элементов списков:
 - a. `a[i] + b[i]` (поэлементное сложение двух списков)
3. `operator+` (число) — добавление константы:
 - a. Добавляет одно и то же число ко всем элементам списка
4. `operator--` (префиксный/постфиксный декремент) — переход к предыдущему элементу:
 - a. Реализуется с помощью класса-итератора
 - b. Итератор должен уметь двигаться назад по списку

Анализ задачи:

1. Создаем связный список (или динамический массив) целых чисел.
2. Реализуем свой класс-итератор для обхода списка.
3. Перегружаем операторы: `[]` для доступа по индексу, `+` для сложения списков и добавления числа, `--` для движения итератора назад.
4. В `main()` показываем работу всех операций.

Код:

```
#include <iostream>

using namespace std;

// СТРУКТУРА УЗЛА СПИСКА
struct Node {
    int data;          // Данные (ключ)
    Node* next;        // Указатель на следующий узел
    Node* prev;        // Указатель на предыдущий узел (для движения назад)

    Node(int val) : data(val), next(nullptr), prev(nullptr) {}
};

// КЛАСС-ИТЕРАТОР
class Iterator {
private:
    Node* current;     // Текущий узел
    Node* tail;        // Хвост списка (нужен для корректной работы -- от end())

public:
    // Конструктор
    Iterator(Node* node = nullptr, Node* listTail = nullptr)
        : current(node), tail(listTail) {}

    // Разыменование итератора (получение значения)
    int& operator*() {
        return current->data;
    }

    // Префиксный инкремент (++it) - переход к следующему элементу
    Iterator& operator++() {
        if (current) current = current->next;
        return *this;
    }

    // Постфиксный инкремент (it++)
```

```

Iterator operator++(int) {
    Iterator temp = *this;
    ++(*this);
    return temp;
}

// Префиксный декремент (--it) - переход к предыдущему элементу
Iterator& operator--() {
    if (current) {
        current = current->prev;
    }
    else if (tail) {
        // Если current == nullptr (end()), переходим к последнему элементу
        current = tail;
    }
    return *this;
}

// Постфиксный декремент (it--)
Iterator operator--(int) {
    Iterator temp = *this;
    --(*this);
    return temp;
}

// Операторы сравнения
bool operator==(const Iterator& other) const {
    return current == other.current;
}

bool operator!=(const Iterator& other) const {
    return current != other.current;
}

// Проверка на конец списка
bool isEnd() const {
    return current == nullptr;
}

```

```
};
```

```
// КЛАСС-КОНТЕЙНЕР: СПИСОК
```

```
class List {
```

```
private:
```

```
    Node* head;    // Голова списка  
    Node* tail;    // Хвост списка  
    int size;      // Количество элементов
```

```
public:
```

```
    // КОНСТРУКТОРЫ
```

```
    // 1. Конструктор по умолчанию
```

```
List() : head(nullptr), tail(nullptr), size(0) {  
    cout << "Конструктор по умолчанию List\n";  
}
```

```
    // 2. Конструктор с параметром (создание списка из n элементов)
```

```
List(int n) : head(nullptr), tail(nullptr), size(0) {  
    cout << "Конструктор с параметром List(" << n << ")\n";  
    for (int i = 0; i < n; i++) {  
        add(i * 10); // Добавляем элементы 0, 10, 20, 30...  
    }  
}
```

```
    // 3. Конструктор копирования
```

```
List(const List& other) : head(nullptr), tail(nullptr), size(0) {  
    cout << "Конструктор копирования List\n";  
    Node* current = other.head;  
    while (current) {  
        add(current->data);  
        current = current->next;  
    }  
}
```

```
    // ДЕСТРУКТОР
```

```
~List() {  
    cout << "Деструктор List\n";
```

```

        clear();
    }

// ВСПОМОГАТЕЛЬНЫЕ МЕТОДЫ

// Добавить элемент в конец списка
void add(int value) {
    Node* newNode = new Node(value);

    if (!head) {
        // Список пуст
        head = tail = newNode;
    }
    else {
        // Добавляем в конец
        tail->next = newNode;
        newNode->prev = tail;
        tail = newNode;
    }
    size++;
}

// Очистка списка
void clear() {
    Node* current = head;
    while (current) {
        Node* temp = current;
        current = current->next;
        delete temp;
    }
    head = tail = nullptr;
    size = 0;
}

// Получить размер списка
int getSize() const {
    return size;
}

```

```

// Получить итератор на начало
Iterator begin() {
    return Iterator(head, tail); // Передаём tail
}

// Получить итератор на конец
Iterator end() {
    return Iterator(nullptr, tail); // Передаём tail для обработки --
}

// 1. ОПЕРАТОР [] – ДОСТУП ПО ИНДЕКСУ
int& operator[](int index) {
    if (index < 0 || index >= size) {
        cout << "Ошибка: индекс за пределами списка!\n";
        static int dummy = 0;
        return dummy;
    }

    Node* current = head;
    for (int i = 0; i < index; i++) {
        current = current->next;
    }
    return current->data;
}

// 2. ОПЕРАТОР + (ВЕКТОР) – СЛОЖЕНИЕ СПИСКОВ
List operator+(const List& other) const {
    List result;
    Node* p1 = head;
    Node* p2 = other.head;

    // Складываем пока есть элементы в обоих списках
    while (p1 && p2) {
        result.add(p1->data + p2->data);
        p1 = p1->next;
        p2 = p2->next;
    }
}

```

```

        // Добавляем оставшиеся элементы первого списка
        while (p1) {
            result.add(p1->data);
            p1 = p1->next;
        }

        // Добавляем оставшиеся элементы второго списка
        while (p2) {
            result.add(p2->data);
            p2 = p2->next;
        }

        return result;
    }

// 3. ОПЕРАТОР + (ЧИСЛО) – ДОБАВЛЕНИЕ КОНСТАНТЫ
List operator+(int value) const {
    List result;
    Node* current = head;

    while (current) {
        result.add(current->data + value);
        current = current->next;
    }

    return result;
}

// 4. ОПЕРАТОР ПРИСВАИВАНИЯ
List& operator=(const List& other) {
    if (this != &other) { // Проверка на самоприсваивание
        clear(); // Очищаем текущий список

        Node* current = other.head;
        while (current) {
            add(current->data);
            current = current->next;
        }
    }
}

```



```

        }
    }
    cout << "Оператор присваивания List\n";
    return *this;
}

// ДРУЗЬЯ ДЛЯ ВВОДА/ВЫВОДА
friend ostream& operator<<(ostream& out, const List& list);
friend istream& operator>>(istream& in, List& list);
};

// ОПЕРАТОР ВЫВОДА
ostream& operator<<(ostream& out, const List& list) {
    out << "[";
    Node* current = list.head;

    while (current) {
        out << current->data;
        if (current->next) out << ", ";
        current = current->next;
    }

    out << "];";
    return out;
}

// ОПЕРАТОР ВВОДА
istream& operator>>(istream& in, List& list) {
    int n, value;
    cout << "Введите количество элементов: ";
    in >> n;

    list.clear();
    cout << "Введите " << n << " элементов:\n";
    for (int i = 0; i < n; i++) {
        in >> value;
        list.add(value);
    }
}

```

```

    return in;
}

int main() {
    setlocale(LC_ALL, "ru");

    cout << "1. КОНСТРУКТОРЫ\n\n";

    // Конструктор по умолчанию
    List list1;
    cout << "list1 (пустой): " << list1 << "\n\n";

    // Конструктор с параметром
    List list2(5); // Список из 5 элементов: 0, 10, 20, 30, 40
    cout << "list2 (5 элементов): " << list2 << "\n\n";

    // Конструктор копирования
    List list3 = list2;
    cout << "list3 (копия list2): " << list3 << "\n\n";

    cout << "2. ОПЕРАТОР [] – ДОСТУП ПО ИНДЕКСУ\n\n";

    cout << "list2[0] = " << list2[0] << "\n";
    cout << "list2[2] = " << list2[2] << "\n";
    cout << "list2[4] = " << list2[4] << "\n\n";

    // Изменение элемента
    list2[1] = 100;
    cout << "После list2[1] = 100:\n";
    cout << "list2 = " << list2 << "\n\n";

    cout << "3. ОПЕРАТОР + (ВЕКТОР) – СЛОЖЕНИЕ СПИСКОВ\n\n";

    List list4(3); // [0, 10, 20]
    List list5(3); // [0, 10, 20]

    cout << "list4 = " << list4 << "\n";

```

```

cout << "list5 = " << list5 << "\n";

List sum = list4 + list5;
cout << "list4 + list5 = " << sum << "\n\n";

cout << "=== 4. ОПЕРАТОР + (ЧИСЛО) - ДОБАВЛЕНИЕ КОНСТАНТЫ ===\n\n";

cout << "list4 = " << list4 << "\n";
List added = list4 + 50;
cout << "list4 + 50 = " << added << "\n\n";

cout << "=== 5. ОПЕРАТОР ПРИСВАИВАНИЯ ===\n\n";

List list6;
cout << "list6 до присваивания: " << list6 << "\n";
list6 = list4;
cout << "list6 после list6 = list4: " << list6 << "\n\n";

cout << "=== 6. ИТЕРАТОР И ОПЕРАТОР -- (ДВИЖЕНИЕ НАЗАД) ===\n\n";

List list7(5); // [0, 10, 20, 30, 40]
cout << "list7: " << list7 << "\n\n";

// Получаем итератор на конец и двигаемся назад
Iterator it = list7.end();

cout << "Движение итератором назад:\n";
--it; // Переход к последнему элементу (40)
cout << "После --it: " << *it << "\n";

--it; // 30
cout << "После --it: " << *it << "\n";

--it; // 20
cout << "После --it: " << *it << "\n";

// Постфиксный декремент
Iterator it2 = it--;

```

```

cout << "\nПостфиксный it--:\n";
cout << "it2 (старое значение): " << *it2 << "\n";
cout << "it (новое значение): " << *it << "\n\n";

cout << "7. ОБХОД СПИСКА ИТЕРАТОРОМ\n\n";

cout << "Прямой обход (вперёд):\n";
for (Iterator it = list7.begin(); !it.isEnd(); ++it) {
    cout << *it << " ";
}
cout << "\n\n";

cout << "Обратный обход (назад):\n";
Iterator itEnd = list7.end();
--itEnd; // Переход к последнему элементу

for (Iterator it = itEnd; true; --it) {
    cout << *it << " ";
    if (it == list7.begin()) break;
}
cout << "\n\n";

cout << "8. ВВОД С КЛАВИАТУРЫ\n\n";

List list8;
cout << "Введите элементы для list8:\n";
cin >> list8;
cout << "Вы ввели: " << list8 << "\n\n";

cout << "9. ИТОГОВЫЙ ВЫВОД\n\n";
cout << "list1 = " << list1 << "\n";
cout << "list2 = " << list2 << "\n";
cout << "list3 = " << list3 << "\n";
cout << "list4 = " << list4 << "\n";
cout << "list5 = " << list5 << "\n";
cout << "list6 = " << list6 << "\n";
cout << "list7 = " << list7 << "\n";
cout << "list8 = " << list8 << "\n";

```

```
    return 0;
}
```

Результаты работы:

Консоль отладки Microsoft Visual Studio

1. КОНСТРУКТОРЫ

Конструктор по умолчанию List
list1 (пустой): []

Конструктор с параметром List(5)
list2 (5 элементов): [0, 10, 20, 30, 40]

Конструктор копирования List
list3 (копия list2): [0, 10, 20, 30, 40]

2. ОПЕРАТОР [] - ДОСТУП ПО ИНДЕКСУ

```
list2[0] = 0
list2[2] = 20
list2[4] = 40
```

После list2[1] = 100:
list2 = [0, 100, 20, 30, 40]

3. ОПЕРАТОР + (ВЕКТОР) - СЛОЖЕНИЕ СПИСКОВ

Конструктор с параметром List(3)
Конструктор с параметром List(3)
list4 = [0, 10, 20]
list5 = [0, 10, 20]
Конструктор по умолчанию List
list4 + list5 = [0, 20, 40]

=== 4. ОПЕРАТОР + (ЧИСЛО) - ДОБАВЛЕНИЕ КОНСТАНТЫ ===

```
list4 = [0, 10, 20]
Конструктор по умолчанию List
list4 + 50 = [50, 60, 70]
```

=== 5. ОПЕРАТОР ПРИСВАИВАНИЯ ===

Конструктор по умолчанию List
list6 до присваивания: []
Оператор присваивания List
list6 после list6 = list4: [0, 10, 20]

=== 6. ИТЕРАТОР И ОПЕРАТОР -- (ДВИЖЕНИЕ НАЗАД) ===

Конструктор с параметром List(5)
list7: [0, 10, 20, 30, 40]

Движение итератором назад:
После --it: 40
После --it: 30
После --it: 20

Постфиксный it--:
it2 (старое значение): 20
it (новое значение): 10

7. ОБХОД СПИСКА ИТЕРАТОРОМ

Прямой обход (вперёд):
0 10 20 30 40

Обратный обход (назад):
40 30 20 10 0

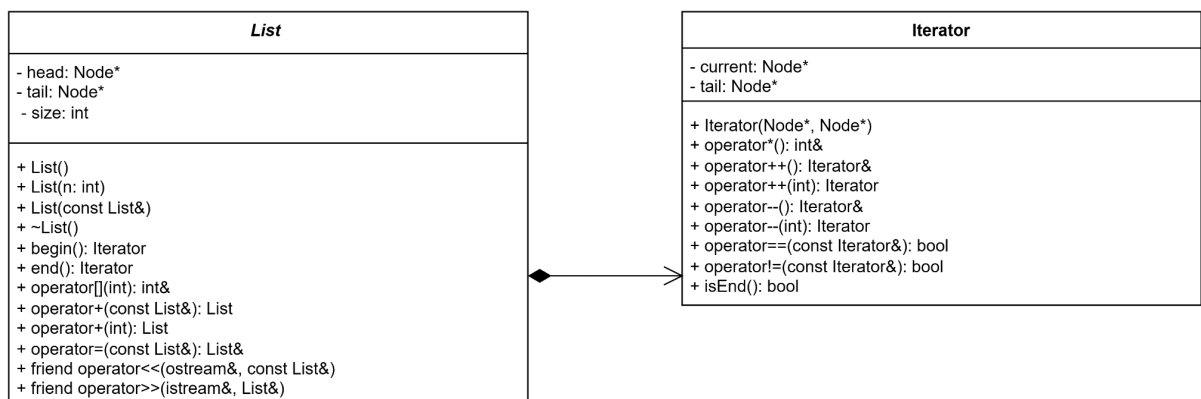
8. ВВОД С КЛАВИАТУРЫ

```
Конструктор по умолчанию List
Введите элементы для list8:
Введите количество элементов: 3
Введите 3 элементов:
0 83 4
Вы ввели: [0, 83, 4]
```

9. ИТОГОВЫЙ ВЫВОД

```
list1 = []
list2 = [0, 100, 20, 30, 40]
list3 = [0, 10, 20, 30, 40]
list4 = [0, 10, 20]
list5 = [0, 10, 20]
list6 = [0, 10, 20]
list7 = [0, 10, 20, 30, 40]
list8 = [0, 83, 4]
Деструктор List
Деструктор List
Деструктор List
Деструктор List
Деструктор List
Деструктор List
Деструктор List
Деструктор List
Деструктор List
Деструктор List
```

UML:



Вопросы:

1. АТД – тип данных, определяемый через операции безотносительно к представлению. Пример: класс в C++.
2. Использование параметров там, где это имеет смысл (например, конструктор с параметром размера).

3. Операции представляются как часть типа (методы класса).
4. Набор однотипных элементов. Примеры: массив, стек, очередь, список, вектор.
5. Доступ к элементам, добавление/удаление, поиск, объединение, специальные.
6. Прямой (по индексу), ассоциативный (по ключу), последовательный (с помощью итератора).
7. Объект, обеспечивающий последовательный доступ к элементам контейнера.
8. Как часть класса-контейнера (методы `first`, `last`, `next`) или как отдельный класс с интерфейсом указателя.
9. Сцепление, объединение упорядоченных, объединение множеств, пересечение, вычитание, извлечение части.
10. Ассоциативный доступ.
11. Стек.
12. `int mas[100]`.
13. `int mas`.
14. Прямой доступ.
15. Последовательный доступ.